Embodied Task Planning via Graph-Informed Action Generation with Large Language Model

Xiang Li[1,2], Ning Yan[2], Masood Mortazavi[2] [1]Purdue University [2]Futurewei Technologies

1. Introduction & Motivation

Background: LLMs have become powerful semantic planners for embodied agents, capable of inferring complex, underspecified human instructions and decomposing them into subgoals. However, in long-horizon tasks, standard planners typically suffer from severe context drift. Because these frameworks rely on linearly growing interaction histories, limited context windows force agents to lose track of high-level goals, leading to repetitive actions and hallucinated, physically impossible transitions

Problem: While external memory is often used to mitigate context limitations, the primary challenge remains the efficient storage and retrieval of relevant state information

Motivation: Embodied agents must maintain strategic coherence over extended periods while dynamically adapting to real-time feedback. While the physical world inherently possesses spatial and temporal order, existing architectures force agents to process information through linear histories or rigid hierarchical trees. This creates a fundamental misalignment between the agent's internal representation and environmental reality. Consequently, there is a critical need for a structured memory that natively captures both ***spatial topology*** and ***temporal state*** transitions.

2. Limitations of Existing Work

1.***Context Drift***: Linearly growing histories of interleaved observations and actions rapidly saturate the context window, causing agents to lose track of high-level goals and generate incoherent plans.

2.***Structure Blocking***: While tree-based decomposition maintains goal coherence, its rigid hierarchy natively blocks concurrent execution, preventing agents from interleaving independent sub-tasks when possible.

3.***Lack of Structured Memory***: Existing agents rely solely on conversation history for reasoning; they lack a structured, cross-episodic memory that allows agents to retrieve and transfer successful structural priors from past experiences.

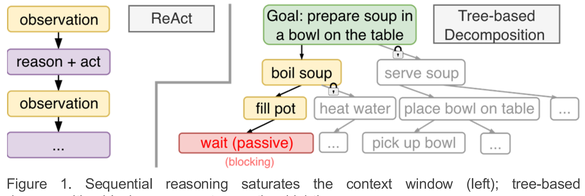


Figure 1. Sequential reasoning saturates the context window (left); tree-based decomposition blocks concurrent execution (right).

3. Our Solution: Graph-In-Graph (GiG)

Key Ideas: raw text observations concatenated with reasoning and action text can easily saturate the context window in long-horizon tasks. Instead of mixing all of them together, decouple the spatial topology and temporal action progress into separate graphs.

1.Scene Graph (Inner Graph): In the scene graph, a node $u \in V_t$ represents an entity (i.e., robot) and an edge $e \in E_t$ represents a relationship between entities (i.e., cheese on-top-of table). The scene graph is passed through a GNN to produce an embedding z_t that captures the topology of the environment structure.
2.State Transition Graph (Outer Graph): In the state transition graph, two scene graph embeddings z_t and z_{t+1} are connected with a directed edge which represents the action a_t that leads to a state transition.
3.Bounded Lookahead (Optional): When the environment transition is available, using the 1-step lookahead as a dynamic verifier to ground the agent's next-step reasoning.
4.Iterative retrieval: Successful experiences are saved to a memory bank $\mathcal{M}$. The agent uses the current scene embedding to query the memory bank for structurally similar priors for the successful execution trace as guidance.

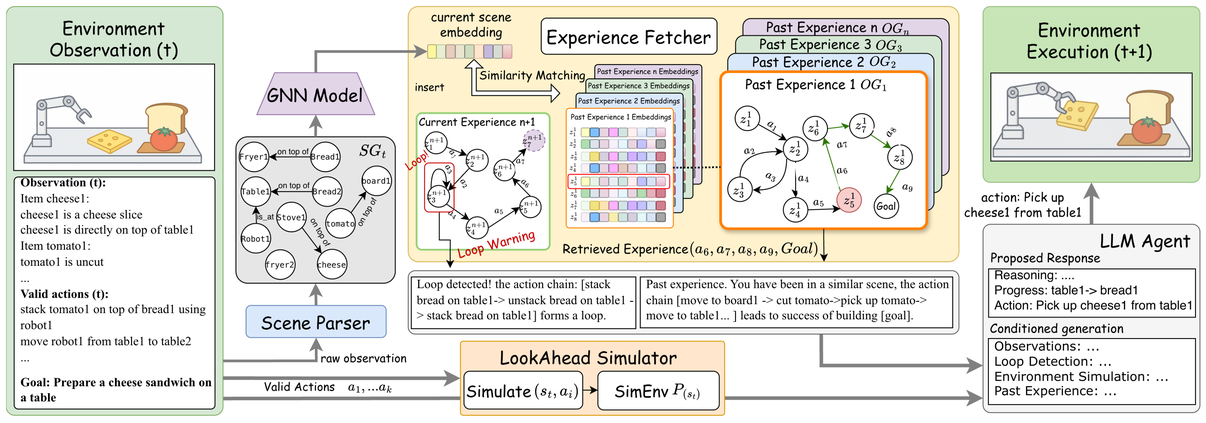
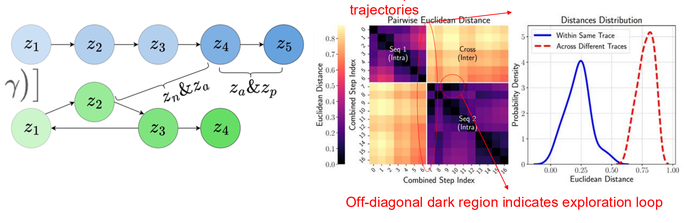


Figure 2. GiG parses the environment observation to build a scene graph, which is encoded by GNN as a structurally rich embedding. This embedding is fed into an experience fetcher to retrieve structurally similar past memory and detect exploration loops. An LLM agent generates the next action conditioned on current observation, past related experience, current goal, and bounded look-ahead results.

4. GNN Encoder Optimization

Assumptions: Physical states evolve gradually, temporally adjacent representations should be more proximal to each other than to randomly sampled states from disjoint trajectories. Based on this assumption, we train the GNN with a composite loss, which is formulated as the weighted sum of a triplet loss and a uniformity loss. For the triplet loss calculation, we adopt the following sampling strategy. The uniformity loss acts as a regularizer to prevent representation collapse.
z_a and z_p: anchor and positive embeddings are sampled from the same sequence and are one step away.
z_n : negative embeddings are sampled from any other sequence in the same batch.

$$\mathcal{L} = \mathcal{L}_{\text{triplet}} + \lambda \mathcal{L}_{\text{uniformity}}$$
$$= \mathbb{E}\left[\max\left(0, \|\mathbf{z}_a - \mathbf{z}_p\|_2^2 - \|\mathbf{z}_a - \mathbf{z}_n\|_2^2 + \gamma\right)\right]$$
$$+\lambda \left(\frac{1}{N^2} \sum_{i=1}^{N-1} \sum_{j=i+1}^{N} \left(\frac{\mathbf{z}_i \cdot \mathbf{z}_j}{\|\mathbf{z}_i\| \|\mathbf{z}_j\|} \right)^2 \right)$$



5. Experiments

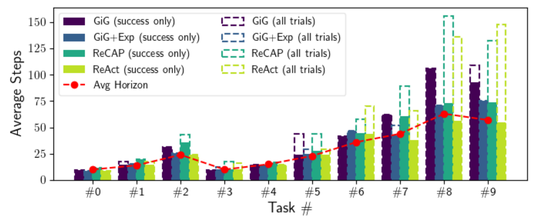
Pass@1 Performance: We compare GiG with other baselines on three embodied planning benchmarks: Robotouille Synchronous, Robotouille Asynchronous, and ALFWorld with backend models of various scales.

Pass@1	Synchronous					Asynchronous				
Model	GiG	GiG(+Exp)	ReCAP	ReAct	CoT	GiG	GiG(+Exp)	ReCAP	ReAct	CoT
Qwen3[1]	93 (+19)	97 (+23)	71	74	7	72 (+37)	82 (+47)	35	31	0
DeepSeek[2]	91 (+19)	88 (+16)	72	53	2	59 (+32)	86 (+59)	27	16	0
Gemini[3]	92 (+0)	90 (-2)	89	92	34	66 (+6)	66 (+6)	21	60	4

[1]Qwen3-235B-A22B-Instruct-2507-FP8; [2]DeepSeek-R1; [3]Gemini-2.5-Flash.

Model	GiG	ReCAP	ExpeL	ReAct
Qwen3[1]	**97**(+6)	89	91	61
DeepSeek[2]	**97**(+15)	82	75	N/A[4]
Gemini[3]	**91**(+2)	86	89	N/A

Execution Efficiency: On easy tasks, GiG takes fewer steps to achieve the goal. On harder tasks, GiG completes tasks that all baselines fail entirely, justifying the modest increase in average steps.



Robustness under Noise: perception noise frequently occurs. we systematically introduce perception noise by injecting noise into the raw text observation to test GiG's robustness:

Pass@1	no-noise	noise/10 steps	noise/5 steps
GiG	93	94	93
ReAct	74	67 (-7)	62 (-12)
ReCAP	71	66 (-5)	62 (-9)

Successful experiences guide Small Models: The successful trajectories collected from large models can improve the performance of smaller models.

Model	GiG	GiG+Exp	ReCAP	ReAct
Qwen3-30B[5]	27	**42**(+15)	19	28
Gemini-2.5-Flash-Lite[6]	19	**26**(+7)	20	20

Context Window Efficiency: GiG consumes roughly 30% of ReCAP's and 21% of ReAct's prefill budget by maintaining only the dynamically aggregated scene graph.

Avg. Context Consumption per Step (Prefill)			
	GiG	ReCAP	ReAct
Tokens	12297 (±1840)	40720 (±26655)	57357 (±46860)

6. Conclusions

Structural Necessity: Decoupling spatial topology from temporal progression can help mitigate the context drift of long-horizon task and enable concurrent task execution.
Perception Resilience: Dynamic graphs aggregation act as self-correcting state representations, successfully neutralizing transient perception noise.
Small Model Enhancement: Structured experience retrieval serves as a model-agnostic plug-in, using it as priors help improve the performance of less capable LLMs.
Systems Efficiency: Maintaining a compact, graph-based context can reduce computational overhead by orders of magnitude compared to interleaved linear-history.

ICML International Conference On Machine Learning

SCAN ME